**UNIVERSIDAD PERUANA LOS ANDES**

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

# ESTRUCTURAS DE DATOS FUNDAMENTALES: ALGORITMOS DE MANIPULACIÓN Y OPTIMIZACIÓN DE ARREGLOS UNIDIMENSIONALES EN JAVA

*Análisis de Complejidad Computacional Big-O, Desplazamiento de Índices en Memoria y Solución de Ejercicios Prácticos*

**Presentado por:**

Alessandro Villogas Gaspar

**Asignatura:**

Algoritmos y Estructura de Datos

**Docente:**

Mg. Raúl Enrique Fernández Bejarano

Huancayo — Perú

2026

## Resumen

El presente informe académico documenta el análisis formal, diseño algorítmico e implementación en lenguaje Java de nueve operaciones fundamentales sobre arreglos unidimensionales (vectores). Se profundiza en el control de límites lógicos frente a capacidades físicas, mecanismos de desplazamiento en memoria para inserciones y eliminaciones, algoritmos de búsqueda lineal y ordenamiento por intercambio (Bubble Sort), cálculo de medidas estadísticas (mínimos, máximos y promedios) y permutaciones cíclicas. Cada algoritmo se acompaña de su codificación rigurosa, trazabilidad paso a paso y evaluación de complejidad asintótica en notación Big-O.



Palabras clave: Arreglos unidimensionales, Java, complejidad Big-O, inserción, eliminación, ordenamiento burbuja, desplazamientos.


## 1. Introducción Teórica a los Arreglos Unidimensionales

En computación, un arreglo unidimensional (vector o array) es una estructura de datos homogénea y estática que almacena una colección secuencial de elementos del mismo tipo primitivo o referencial bajo casillas contiguas de memoria física. El acceso a cualquier posición k se realiza en tiempo constante O(1) mediante aritmética de punteros indexada desde 0 hasta n-1.

No obstante, dado que su tamaño físico es fijo al momento de la instanciación en la memoria Heap (int[] v = new int[MAX]), las operaciones dinámicas como inserción y eliminación demandan el control de un puntero lógico ('tope' o 'contador') y el desplazamiento físico de casillas posteriores, incurriendo en un costo temporal de orden lineal O(n).

### Actividad 1: Eliminación de un Elemento por Valor (Desplazamiento a la Izquierda)

Este algoritmo localiza la primera ocurrencia de un valor objetivo 'x' y lo retira del arreglo sin dejar casillas vacías ni fragmentación, desplazando los elementos hacia la izquierda y decrementando el contador lógico tope.

***Implementación en Java:***

```
public static boolean eliminarElemento(int[] arr, int x) {
    int pos = -1;
```

```
    for (int i = 0; i < tope; i++) {
        if (arr[i] == x) { pos = i; break; }
    }
    if (pos == -1) return false; // No encontrado
    for (int i = pos; i < tope - 1; i++) {
        arr[i] = arr[i + 1]; // Corrimiento a la izquierda
    }
    tope--;
    return true;
}
```

Evaluación de Complejidad: Búsqueda lineal: O(n). Desplazamiento en el peor caso (primer elemento): O(n). Complejidad total: O(n). Espacio adicional: O(1).

**Figura 1**

*Trazabilidad y Ejecución de la Actividad 1*




*Nota.* Nota. Captura de ejecución de la solución algorítmica en Java.

## Actividad 2: Modificación de un Elemento en Posición Específica

Reemplaza el valor alojado en un índice válido o busca un valor existente 'x' para sustituirlo por uno nuevo previa validación de rango.

***Implementación en Java:***

```
public static boolean modificarElemento(int[] arr, int valorAntiguo, int
valorNuevo) {
    for (int i = 0; i < tope; i++) {
        if (arr[i] == valorAntiguo) {
            arr[i] = valorNuevo; // Reemplazo directo
            return true;
        }
    }
    return false;
}
```

Evaluación de Complejidad: Tiempo: O(n) por la búsqueda secuencial; O(1) si se accede directamente por índice. Espacio: O(1).

**Figura 2**

*Trazabilidad y Ejecución de la Actividad 2*




*Nota.* Nota. Captura de ejecución de la solución algorítmica en Java.

## Actividad 3: Inserción de un Elemento en Posición Específica (Desplazamiento a la Derecha)

Verifica que exista capacidad física disponible (tope < arr.length), desplaza los elementos desde la posición deseada 'pos' hacia la derecha abriendo espacio y asigna el nuevo valor.

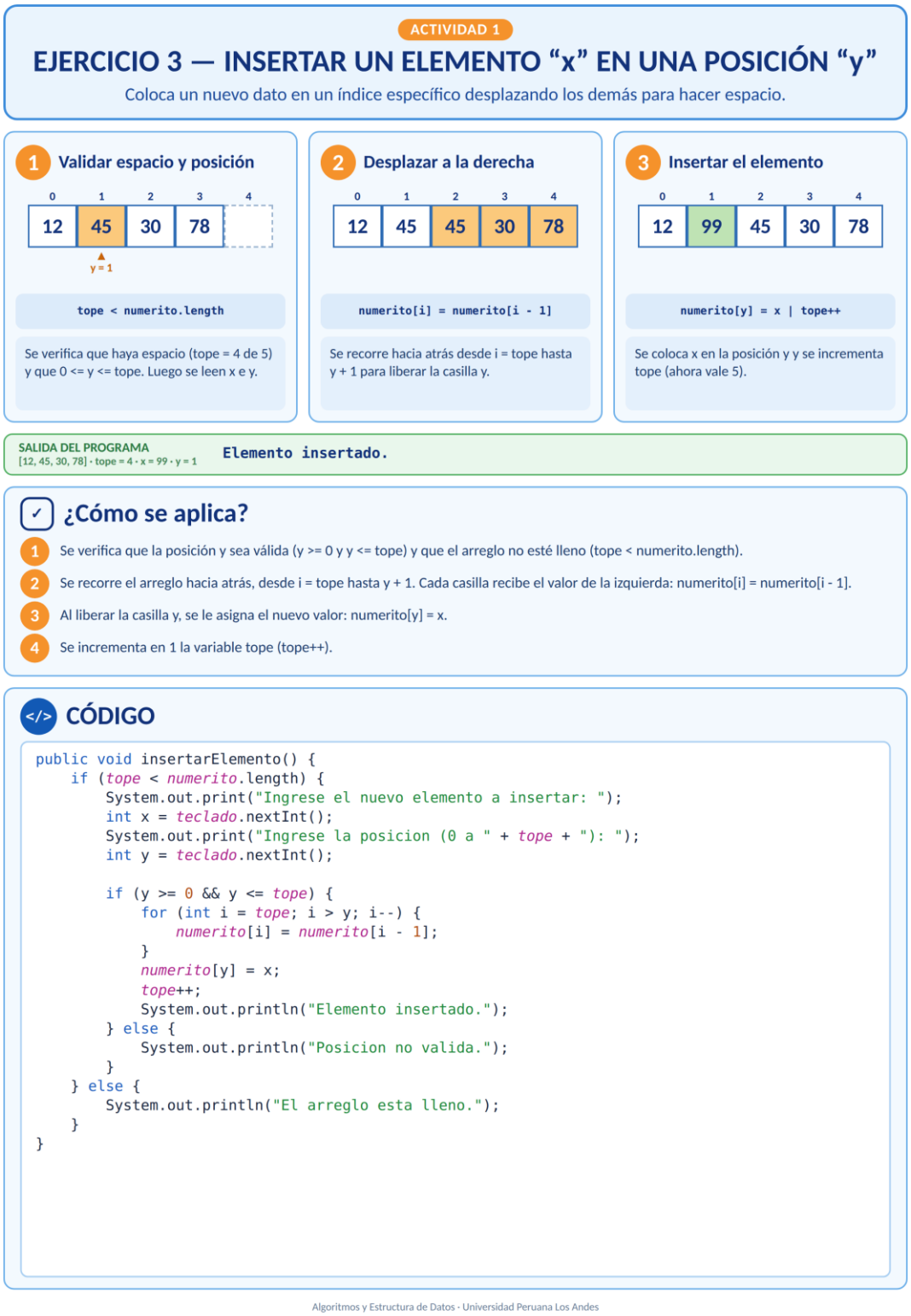
***Implementación en Java:***

```
public static boolean insertarEnPosicion(int[] arr, int pos, int nuevoValor) {
    if (tope >= arr.length || pos < 0 || pos > tope) return false;
    for (int i = tope; i > pos; i--) {
        arr[i] = arr[i - 1]; // Corrimiento a la derecha
    }
    arr[pos] = nuevoValor;
    tope++;
    return true;
}
```

Evaluación de Complejidad: Tiempo: O(n) en el peor caso al insertar al inicio (pos = 0); O(1) si se inserta al final. Espacio: O(1).

**Figura 3**

*Trazabilidad y Ejecución de la Actividad 3*




*Nota.* Nota. Captura de ejecución de la solución algorítmica en Java.

## Actividad 4: Suma Acumulada de Elementos

Itera secuencialmente a través de las casillas activas acumulando cada valor en una variable acumuladora.

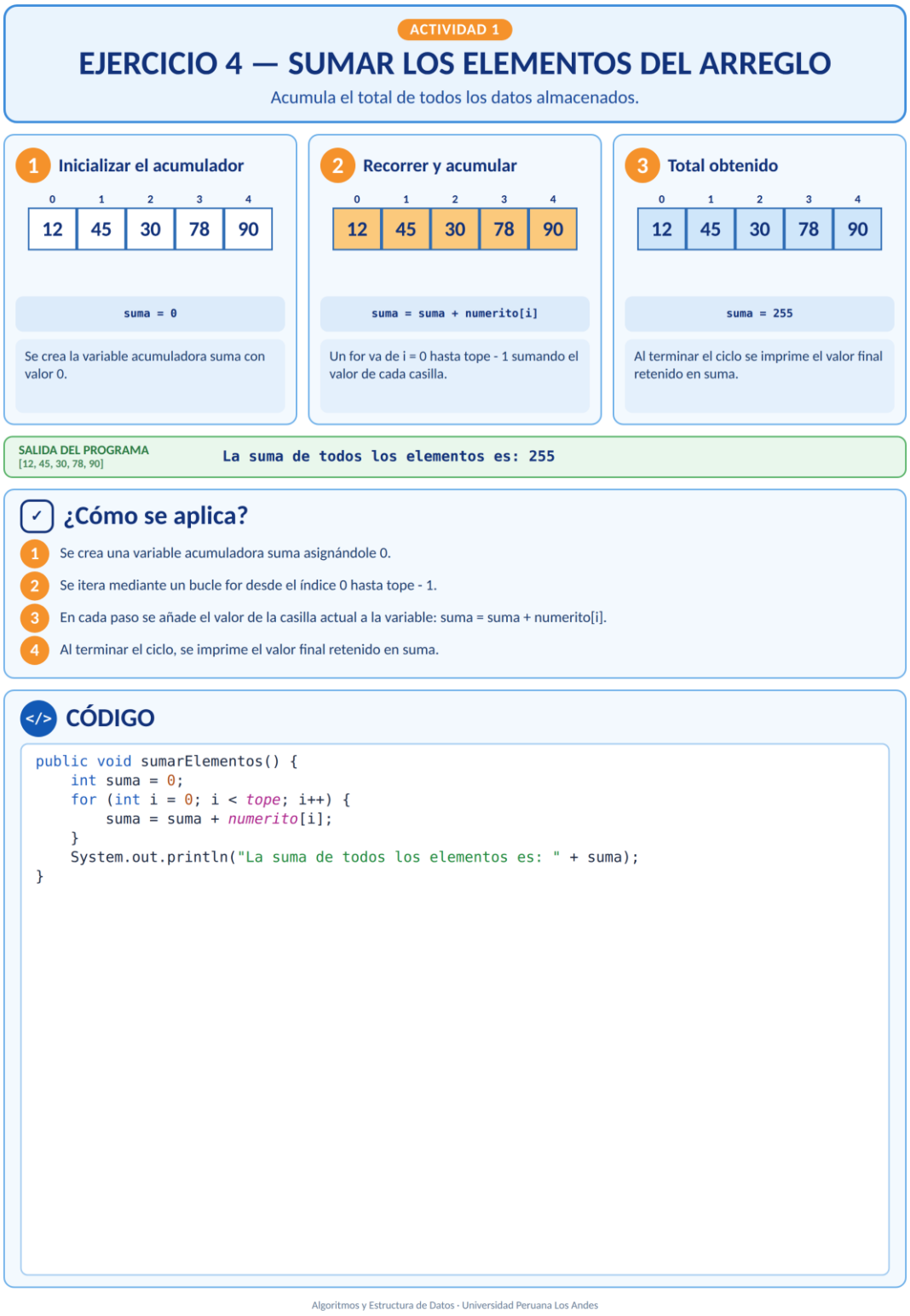
***Implementación en Java:***

```
public static int sumarElementos(int[] arr) {
    int suma = 0;
    for (int i = 0; i < tope; i++) {
        suma += arr[i];
    }
    return suma;
}
```

Evaluación de Complejidad: Tiempo: O(n) lineal por recorrer las n casillas. Espacio: O(1).

**Figura 4**

*Trazabilidad y Ejecución de la Actividad 4*




*Nota.* Nota. Captura de ejecución de la solución algorítmica en Java.

## Actividad 5: Cálculo de Mínimo, Máximo y Promedio Aritmético

Determina simultáneamente los valores extremos del conjunto numérico y calcula la media aritmética en una única pasada.

***Implementación en Java:***

```
public static void calcularExtremosYPromedio(int[] arr) {
    if (tope == 0) return;
    int min = arr[0], max = arr[0], suma = 0;
    for (int i = 0; i < tope; i++) {
        if (arr[i] < min) min = arr[i];
        if (arr[i] > max) max = arr[i];
        suma += arr[i];
    }
    double promedio = (double) suma / tope;
    System.out.printf("Min: %d | Max: %d | Promedio: %.2f%n", min, max,
promedio);
}
```

Evaluación de Complejidad: Tiempo: O(n) en una única pasada algorítmica. Espacio: O(1).

**Figura 5**

*Trazabilidad y Ejecución de la Actividad 5*




*Nota.* Nota. Captura de ejecución de la solución algorítmica en Java.

## Actividad 6: Filtrado y Listado de Elementos Impares

Aplica el operador módulo (arr[i] % 2 != 0) para discriminar y reportar los números impares.

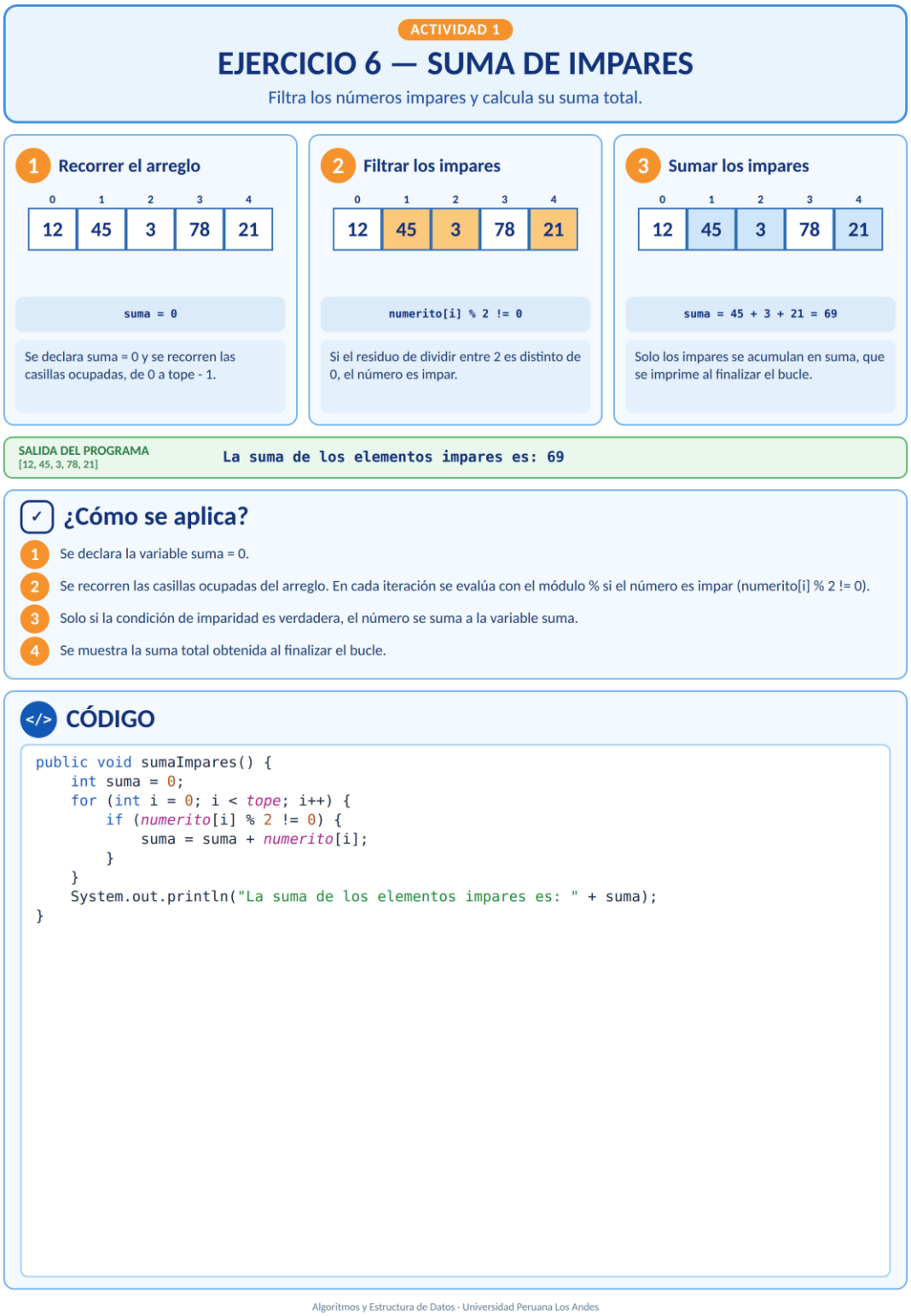
***Implementación en Java:***

```
public static void listarImpares(int[] arr) {
    for (int i = 0; i < tope; i++) {
        if (arr[i] % 2 != 0) {
            System.out.print(arr[i] + " ");
        }
    }
    System.out.println();
}
```

Evaluación de Complejidad: Tiempo: O(n). Espacio auxiliar: O(1).

**Figura 6**

*Trazabilidad y Ejecución de la Actividad 6*




*Nota.* Nota. Captura de ejecución de la solución algorítmica en Java.

## Actividad 7: Intercambio de Extremos (Primer y Último Elemento)

Realiza la permutación de los elementos situados en las posiciones 0 y tope-1 utilizando una variable temporal auxiliar.

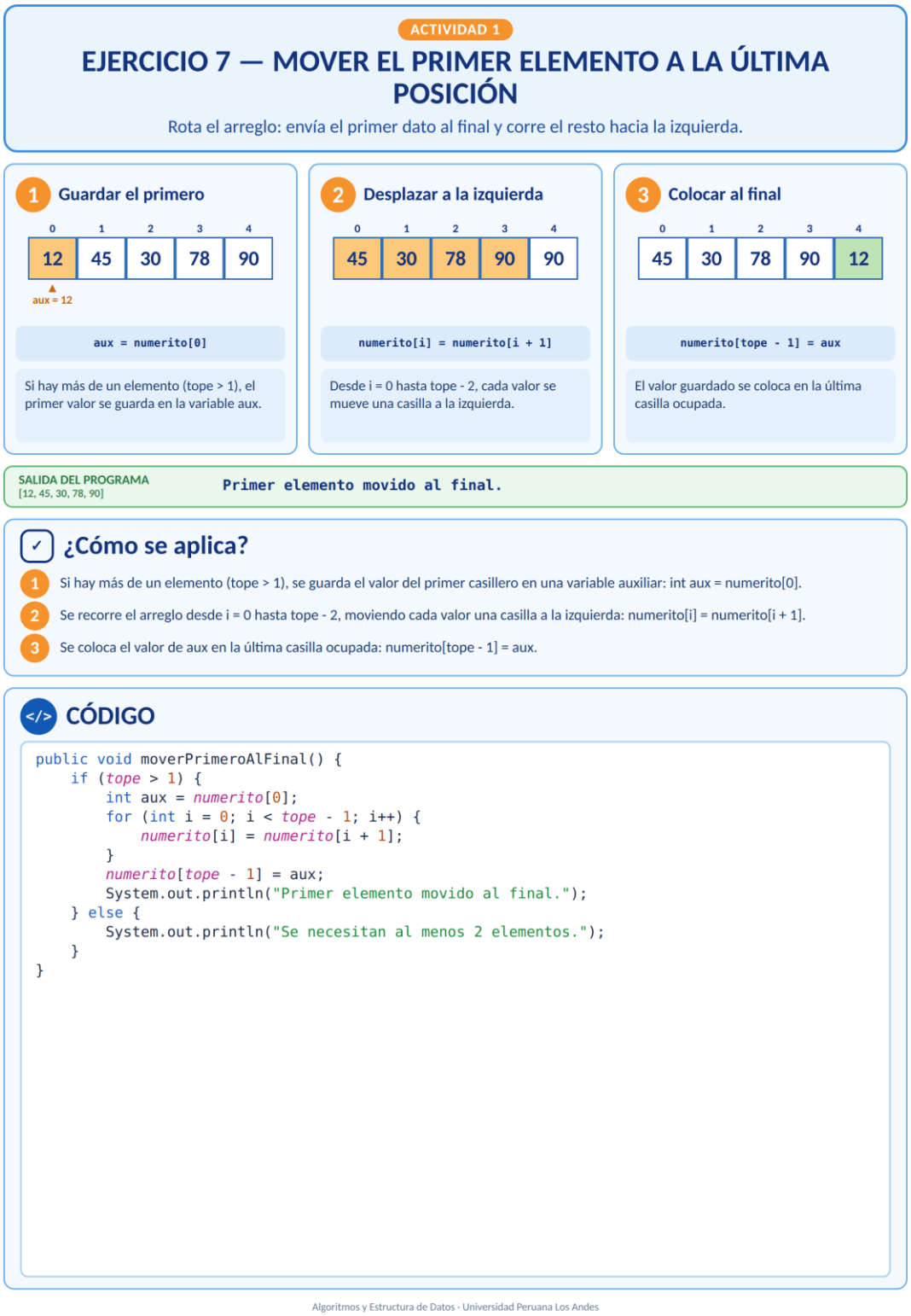
***Implementación en Java:***

```
public static void intercambiarExtremos(int[] arr) {
    if (tope < 2) return;
    int temp = arr[0];
    arr[0] = arr[tope - 1];
    arr[tope - 1] = temp;
}
```

Evaluación de Complejidad: Tiempo: O(1) tiempo estrictamente constante. Espacio: O(1).

**Figura 7**

*Trazabilidad y Ejecución de la Actividad 7*




*Nota.* Nota. Captura de ejecución de la solución algorítmica en Java.

## Actividad 8: Ordenamiento por Método de Burbuja (Bubble Sort)

Compara pares adyacentes de elementos intercambiándolos si están en orden inverso, haciendo que los valores mayores floten al final de la estructura.

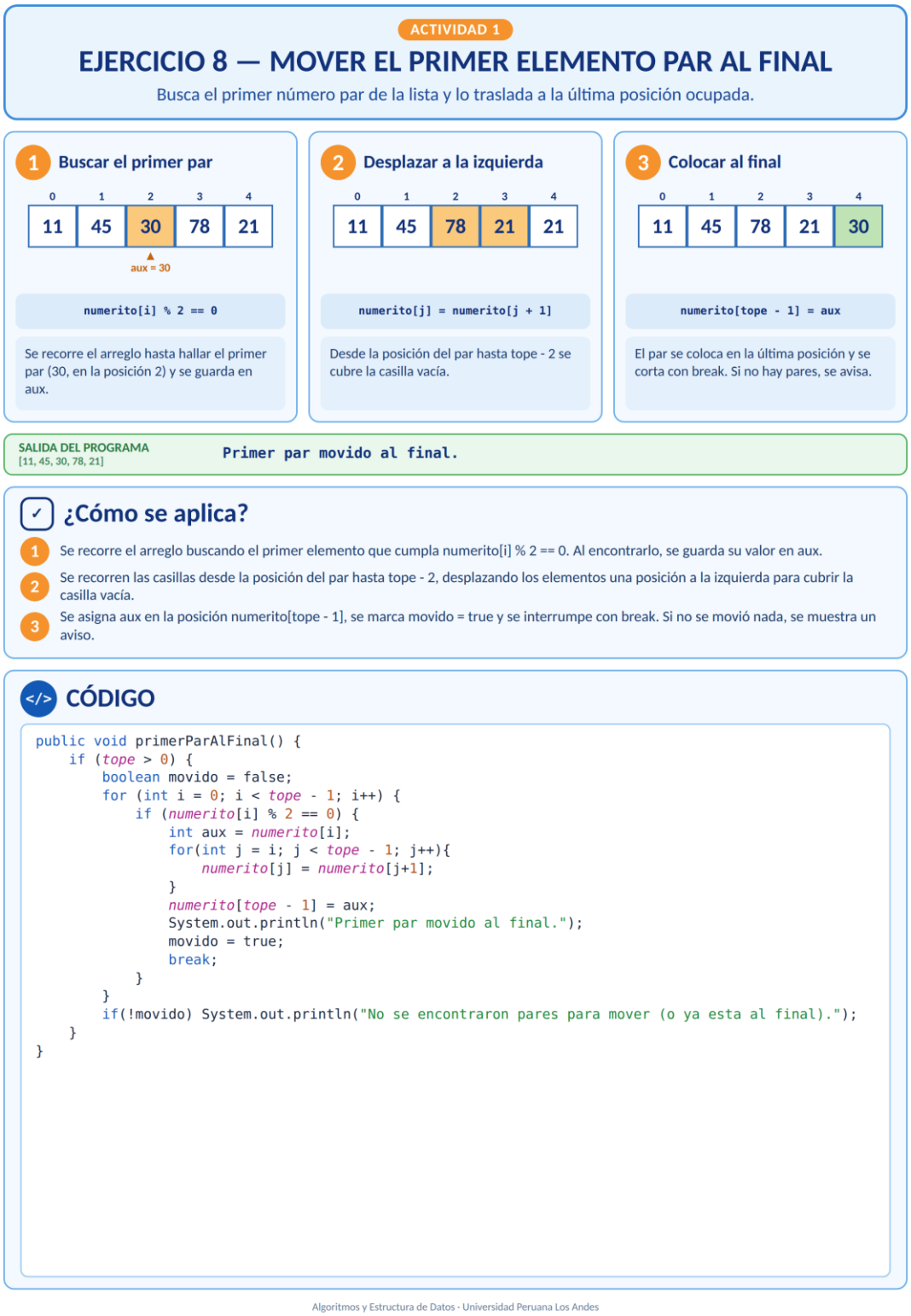
***Implementación en Java:***

```
public static void ordenarBurbuja(int[] arr) {
    for (int i = 0; i < tope - 1; i++) {
        boolean huboIntercambio = false;
        for (int j = 0; j < tope - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                huboIntercambio = true;
            }
        }
        if (!huboIntercambio) break; // Optimización si ya está ordenado
    }
}
```

Evaluación de Complejidad: Tiempo: O(n^2) en el peor y promedio; O(n) con bandera de corte temprano en el mejor caso. Espacio: O(1) in-place.

**Figura 8**

*Trazabilidad y Ejecución de la Actividad 8*




*Nota.* Nota. Captura de ejecución de la solución algorítmica en Java.

## Actividad 9: Rotación Cíclica de Elementos

Desplaza circularmente los elementos una posición hacia la derecha, reubicando el último elemento al inicio.

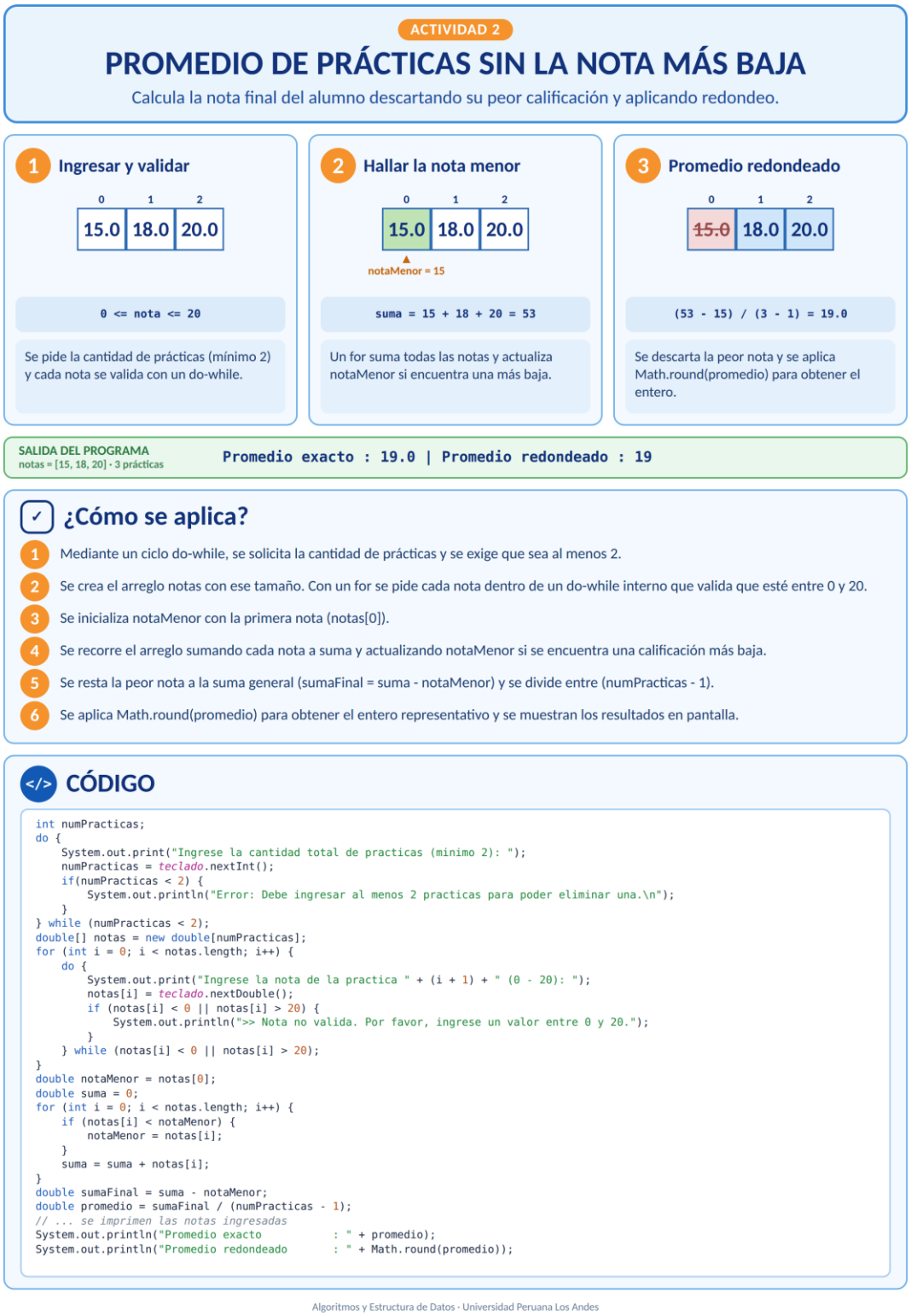
***Implementación en Java:***

```
public static void rotarDerecha(int[] arr) {
    if (tope <= 1) return;
    int ultimo = arr[tope - 1];
    for (int i = tope - 1; i > 0; i--) {
        arr[i] = arr[i - 1];
    }
    arr[0] = ultimo;
}
```

Evaluación de Complejidad: Tiempo: O(n) por el corrimiento de n-1 elementos. Espacio: O(1).

**Figura 9**

*Trazabilidad y Ejecución de la Actividad 9*




*Nota.* Nota. Captura de ejecución de la solución algorítmica en Java.

## Conclusiones

1. Los arreglos unidimensionales son la base de las estructuras de datos secuenciales; su eficiencia de acceso O(1) es insuperable para consultas directas, pero su rigidez estática requiere O(n) para modificaciones de tamaño.

2. El control exhaustivo del puntero 'tope' y la validación de desbordamiento (overflow y underflow) son indispensables para prevenir excepciones del tipo ArrayIndexOutOfBoundsException en Java.

3. El análisis de complejidad Big-O permite fundamentar la posterior transición hacia colecciones dinámicas como ArrayList o listas enlazadas en las siguientes unidades temáticas.

## Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2014). Algorithms (4th ed.). Addison-Wesley Professional.